Analysis & Design Document

LLM Host Failsafe System

Game-Based Learning Study — Game-LLM-Interactive Condition

Valentino Clerx

Fontys Master of Applied IT

2026

1. Introduction

The "Game-LLM-Interactive" condition relies on a continuous connection to an external Large Language Model (LLM) to power the NPC Veterinarian. Dependency on a single third-party API provider introduces a "Single Point of Failure" (SPOF) risk. This document details the design of a redundant hosting architecture, a Host Failsafe, implemented within the Godot engine to ensure high availability and research continuity during live playtesting.

2. System Architecture

The failsafe system follows a Primary-Secondary Fallback pattern. The logic is encapsulated within a central GeminiManager.gd singleton that abstracts the complexities of HTTP request management and error handling from the UI layer.

2.1 Component Breakdown

- **Primary Host (Portkey):** Acts as the main gateway. Portkey is utilized for its advanced logging, caching, and potential for model routing.
- **Secondary Host (Google Gemini Direct)**: Acts as the failsafe. If the primary gateway returns a non-200 status code or fails to respond, the system bypasses the intermediary and calls the Google Generative Language API directly.
- **GeminiManager (The Orchestrator)**: Manages the state of the request and emits a response_received signal only once a successful reply is obtained from either source.

3. Design

3.1 Strategy Pattern for Redundancy

The system utilizes two distinct strategies for content generation. The query() function serves as the entry point, defaulting to Strategy 1.

- **Strategy 1 (Portkey Gateway):** The manager constructs a standard REST request using the x-portkey-api-key. It targets the chat/completions endpoint.
- **Strategy 2 (Direct Google Backup):** If Strategy 1 fails, the system executes _call_google_backup. This uses the v1beta direct Google API endpoint and switches from a "Chat Completion" JSON structure to a "Generate Content" structure.

3.2 Failure Logic & Recursion Prevention

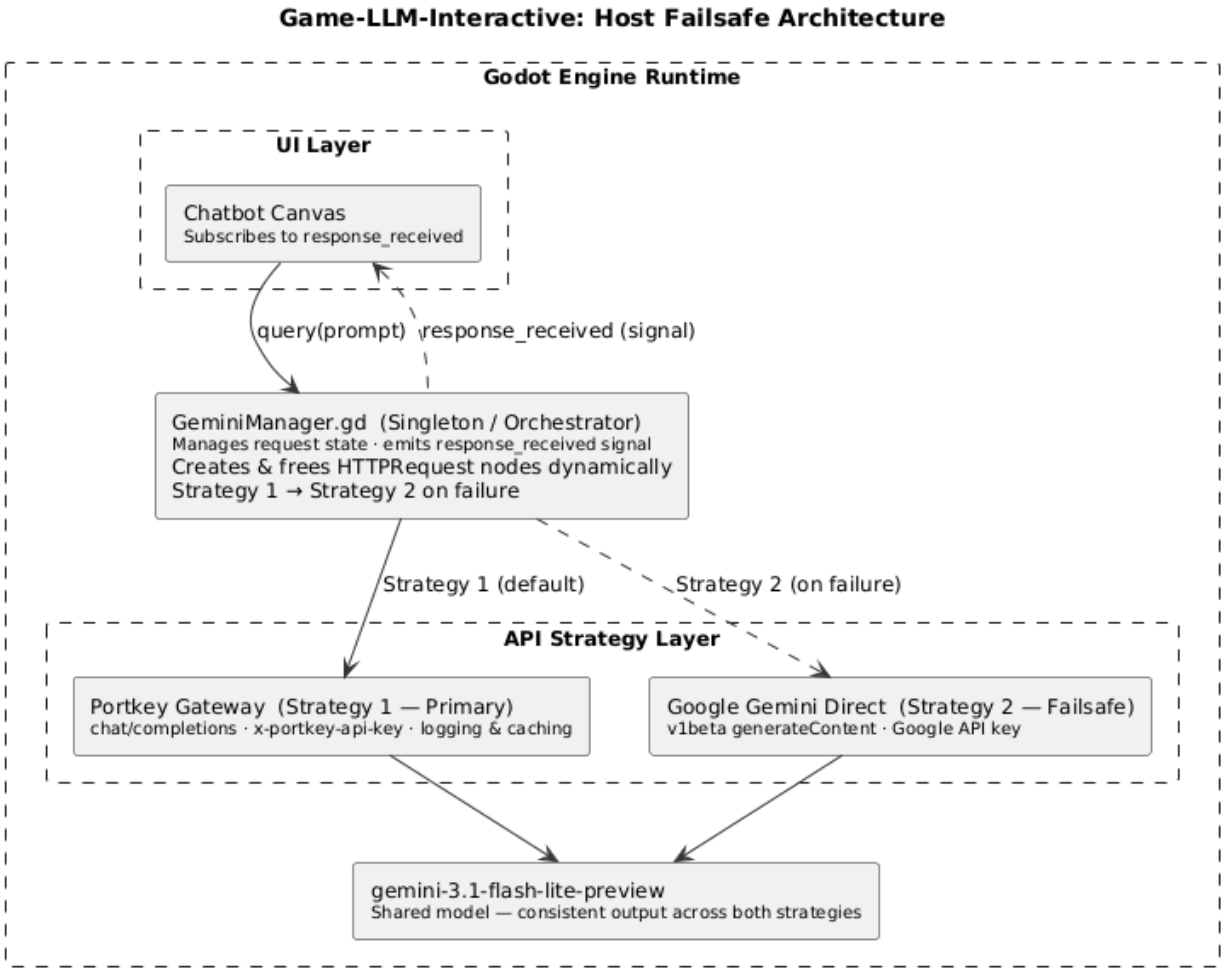
The failsafe is triggered within the _on_portkey_finished callback. By checking the response_code, the system can react to network issues, rate limits, or API outages.

```
GDScript
func _on_portkey_finished(_result, response_code, _headers, body, node, prompt_text):
    # If Primary (Portkey) fails, switch to Strategy 2
    if response_code == 200 and json_res.has("choices"):
        response_received.emit(ai_text)
    else:
        print("Portkey Failed. Switching to Google Backup...")
        _call_google_backup(prompt_text)
```

3.3 Data Consistency

To ensure the player experience remains identical regardless of the host, both strategies utilize the same model version (gemini-3.1-flash-lite-preview). This ensures that the "temperature" and "intelligence" of the NPC remain consistent, preventing technical variance from tainting the experimental research data.

3.4 System Diagram



4. Implementation Details

4.1 Request Management
Because Godot's HTTPRequest nodes are asynchronous, the manager dynamically creates and attaches a new node for every request. These nodes are then automatically cleaned up using node.queue_free() upon completion of either the primary or secondary call to prevent memory leaks.

4.2 Signal Flow
The UI (the Chatbot canvas) only connects to the response_received signal. It is entirely unaware of which host provided the data. This encapsulation allows the engineering team to swap API keys or providers in the backend without requiring changes to the frontend dialogue scripts.

5. Design Considerations

5.1 Security
Currently, API keys are stored as variables within the script. While sufficient for a controlled research environment/pilot run, this is identified as a security risk. For final deployment, these should be moved to an encrypted configuration file or environment variables.

5.2 Latency vs. Reliability
The failsafe introduces a slight latency penalty in the event of a failure (the time taken for the first request to timeout or return an error). However, for a research study, Reliability is prioritized over Latency. A 2-second delay in text generation is preferable to a total system crash during a participant's session.